

Ques: Given N array elements and Q queries on same array.

For each query, calculate sum of all elements in given range

$L \leq R$

arr[10]:- ⁰-3 ¹6 ²2 ³4 ⁴5 ⁵2 ⁶8 ⁷-9 ⁸3 ⁹1

Q=5

L - R sum

4 8 : 9

6 9 : 3

1 3 : 12

0 4 : 14

7 7 : -9

idea:- Can I say that for every query, iterate and get the sum from $[L \rightarrow R]$

for ($i=1; i \leq Q; i++$) { $\rightarrow$ Q queries.

$s=0;$

 for ($j=L; j \leq R; j++$) { $\rightarrow$ in w.c $\rightarrow O(N)$

$s = s + arr[j];$

 }

 print(s);

}

T.C = $O(Q * N)$

S.C = $O(1)$

Overs: 1 2 3 4 5 6 7 8 9 10

Scores: 2 8 14 29 31 49 65 79 88 97

Total runs scored in last over:- $s[10] - s[9]$
 $[10, 10] \uparrow$ 97 - 88 = 9 runs

Total runs from 1st to 5th over:- $s[5]$
 $[1, 5]$ 31 runs

Total runs in last 5 overs:- $s[10] - s[5]$
Runs scored at the end of 10th over = 97
Runs scored at the end of 5th over = 31
 $(97 - 31) = 66$ runs.

Total runs in just 7th over:- $s[7] - s[6]$
 $[7, 7]$ 65 - 49 = 16 runs.

Total runs from 3rd to 6th over:- $s[6] - s[2]$
 $[3, 6]$ 49 - 8 = 41 runs

$$[L-R] = s[R] - s[L-1] \rightarrow O(1)$$

cumulative data $\rightarrow$ sum of all runs till that particular over.

// cumulative data from start = pfSum[]

pfSum[i] = sum of all elements [0, i]

$$\text{pfSum}[3] = \text{sum}[0-3]$$

$$\text{pfSum}[6] = \text{sum}[0-6]$$

$$\text{pfSum}[0] = \text{sum}[0-0] = \text{arr}[0]$$

	0	1	2	3	4	5	6	7	8	9
arr [10]:	-3	6	2	4	5	2	8	-9	3	1
pf [10]:	-3	3	5	9	14	16	24	15	18	19

$$Q_1) [4-8] :- pf[8] = sum[0-8] = sum[0-3] + sum[4-8]$$

$$pf[8] = pf[3]$$

$$sum[4-8] = pf[8] - pf[3]$$

$$18 - 9 = 9$$

$$Q_2) [3-7] :- pf[7] - pf[2]$$

$$Q_3) [i, j] :- pf[j] = sum[0-j] = sum[0, i-1] + sum[i, j]$$

$$pf[j] = pf[i-1] + sum[i, j]$$

$$sum[i, j] = pf[j] - pf[i-1]$$

Edge case

e.g. $\begin{matrix} i & j \\ 0 & 3 \end{matrix} :- pf[3] - pf[0-1] \Rightarrow pf[3] - pf[-1]$ ↪ Index out of bounds exception.

↓

Actual ans. :- $pf[3]$, if $i=0$

$sum[i-1] :-$

```

if (i==0) {
    ans = pf[j];
}
else {
    ans = pf[j] - pf[i-1];
}

```

// How to construct a pf[]

$$\begin{aligned}
 pf[0] &= sum[0-0] = arr[0] \\
 pf[1] &= sum[0-1] = arr[0] + arr[1] \\
 pf[2] &= sum[0-2] = arr[0] + arr[1] + arr[2] \\
 pf[i] &= sum[0,i] = arr[0] + arr[1] + \dots + arr[i-1] + arr[i]
 \end{aligned}$$

$\downarrow$
 $sum[0, i-1]$
 $\downarrow$
 $pf[i-1]$

$$pf[i] = pf[i-1] + arr[i]$$

int pf[N];

pf[0] = arr[0];

for(int i=1; i < N; i++){

pf[i] = pf[i-1] + arr[i];

for(int i=1; i <= Q; i++){

// sum[l-r] or sum[i-j]

if(l==0){

ans = pf[r];

else{

ans = pf[r] - pf[l-1];

print(ans);

$\rightarrow O(N)$

$\rightarrow O(Q)$

Overall T.C = $O(N + Q)$

S.C = $O(N)$ $\downarrow$ optimise.
 $O(1)$

ToDo:- You can modify i/p array to store pf[].

Ques 2 Equilibrium Index

Given N array elements, count no. of equilibrium indices?

An index is said to be equilibrium index if?

$$\begin{array}{l} \text{sum of all elements} \\ \text{Before } i^{\text{th}} \text{ index} \\ \text{sum}[0, i-1] \rightarrow \text{pf}[i-1] \end{array} = \begin{array}{l} \text{sum of all elements} \\ \text{after } i^{\text{th}} \text{ index} \\ \text{sum}[i+1, N-1] \rightarrow \text{pf}[N-1] - \text{pf}[i] \end{array}$$

Note:- if $i=0$, left $= 0$; if $i=N-1$, right $= 0$

e.g. 1. arr [4] =

	0	1	2	3
arr	-3	2	4	-1
left	0	-3	-1	3
right	5	3	-1	0

$\rightarrow \text{ans} = 1$

e.g. 2. arr [7] =

	0	1	2	3	4	5	6
arr	-7	1	5	2	-4	3	0
left	0	-7	-6	-1	1	-3	0
right	7	6	1	-1	3	0	0

$\rightarrow \text{ans} = 2$

e.g. 4. arr [7] =

	0	1	2	3	4	5	6
arr	3	-1	2	-1	1	2	1
left	0	3	2	4	3	4	6
right	4	5	3	4	3	1	0

$\rightarrow \text{ans} = 2$

Brute force:- for every index, check if it is equilibrium or not.

eg.3

arr[3] =	0	1	2
	3	-2	2
left =	0		
right =	0		

→ 0th index can also be an equilibrium index.

Pseudo Code:-

```
int pf[N];
```

```
// construct pf[]
```

```
int count=0;
```

```
for(i=0; i<N; i++){
```

```
// Is ith index an equilibrium index
```

```
left = sum[0, i-1]
```

```
right = sum[i+1, N-1]
```

```
if(left == right){
```

```
    count++;
```

```
}
```

```
}
```

Edge case

if $i=0$, $pf[0-1] = pf[-1]$ X

$pf[i-1] \rightarrow O(1)$

$pf[N-1] - pf[i] \rightarrow O(1)$

T.C = $O(N^2)$

S.C = $O(1)$

Pseudo Code:-

```
int pf[N];
```

```
// construct pf[]
```

```
int count=0;
```

```
for(i=0; i<N; i++){
```

```
// Is ith index an equilibrium index
```

```
if(i==0) left=0
```

```
else left = pf[i-1];
```

```
right = pf[N-1] - pf[i];
```

```
if(left == right){
```

```
    count++;
```

```
}
```

```
}
```

$T.C = O(N + N)$

$= O(2N)$

$T.C = O(N)$

$S.C = O(N) \rightarrow pf[]$

Break :- llpm

Ques 3 Given N array elements. Construct $pfEven[]$ of size N .
 $pfEven[i] = \text{sum of all even index elements from } [0, i]$

e.g. 1

	0	1	2	3	4	5
$arr[6] =$	3	-2	4	6	-3	5
$pfEven[6] =$	3	3	7	7	4	4
$pfEven[0] =$	3					
$pfEven[1] =$	sum of even from $[0-1] = 3$					
$pfEven[2] =$	- - - - -			$[0-2] = 7$		
$pfEven[4] =$	- - - - -			$[0-4] = 4$		
$pfEven[5] =$	- - - - -			$[0-5] = 4$		

e.g. 2

	0	1	2	3	4	5	6	7
$arr[8] =$	2	-1	3	1	4	3	2	-1
$pfEven[8] =$	2	2	5	5	9	9	11	11

(Note: Red arrows in the original image point from index 1 to 2, 3 to 4, and 5 to 6, indicating the cumulative sum of even-indexed elements.)

Pseudo Code:-

```

pfEven[0] = arr[0]
for (i=1; i < N; i++) {
    if (i % 2 == 1) {
        // odd index
        pfEven[i] = pfEven[i-1];
    }
    else { // even index
        pfEven[i] = pfEven[i-1] + arr[i];
    }
}

```

To Do:- Construct $pfOdd[]$ array.

Ques Given N array elements. and Q queries.
for each query you have to calculate sum of all even indices in a given range.

arr[8] = ⁰3 ¹4 ²-2 ³8 ⁴6 ⁵2 ⁶1 ⁷3
 pfEven[] = 3 3 1 1 7 7 8 8

Queries. → Q

[2 6] :- pfEven[6] - pfEven[1] = 5
 8 3

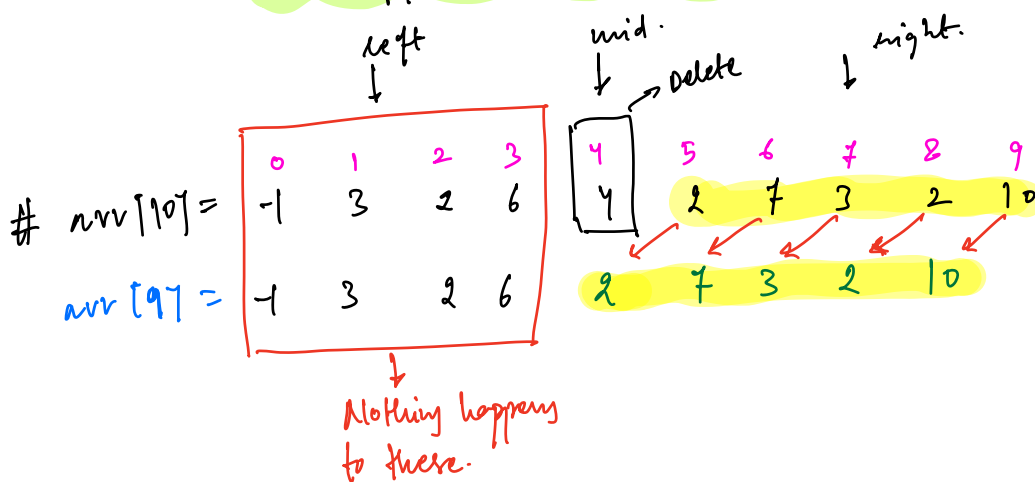
[3 7] :- pfEven[7] - pfEven[2] = 7
 8 1

Query:- sum of all even indices from [i,j]

[i,j] = if i==0
 ans = pfEven[j]

else

ans = pfEven[j] - pfEven[i-1]



Observations:-

- On right side of deleted element,
 - odd index data becomes even index data
 - even index data becomes odd index data.

Special Question $\rightarrow$ Google.

Special Index.

Q6. An index is said to be a special index, if after deleting it,
 $\text{sum of all even indices} = \text{sum of all odd indices}$.
 Count the number of special indices in the given array.

e.g.

	0	1	2	3	4	5
	4	3	2	7	6	-2

Delete index 0

arr[] =

0	1	2	3	4
3	2	7	6	-2

Seven = 8, Sodd = 8

A special index

Delete index = 1

arr[] =

0	1	2	3	4
4	2	7	6	-2

Seven = 9 ; Sodd = 8

X

Delete index = 2

arr[] =

0	1	2	3	4
4	3	7	6	-2

Seven = 9 ; Sodd = 9

✓

Delete index 3

arr[] =

0	1	2	3	4
4	3	2	6	-2

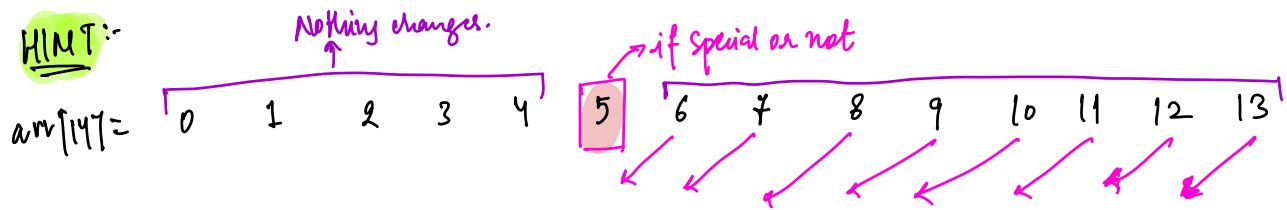
Seven = 4 ; Sodd = 9

Delete index = 4

Delete index = 5

ans = 2

HINT:-



sumEven[0-4] +

sumEven[6-13] → sumOdd[6-13]

sumOdd[0-4] +

sumOdd[6-13] → sumEven[6-13]

total sum of even indices == total sum of odd indices.

sumEven[0-4] +
sumOdd[6-13]

sumOdd[0-4] +
sumEven[6-13]

This is the $\heartsuit$ of this problem.

arr[N] = 0 1 2 3 ... i-1 i i+1 ... N-2 N-1

SumEven[0,i]
SumOdd[0,i]

special or not.

SumEven[i+1, N-1] → oddSum

SumOdd[i+1, N-1] → EvenSum

Teven = SumEven[0,i] + odd[i+1, N-1]

Toad = SumOdd[0,i] + Even[i+1, N-1]

Pseudo Code

```
// construct pfEven[]  
// construct pfOdd[]
```

```
int cnt = 0;
```

```
for (i = 0; i < N; i++) {
```

```
    SEven[0, i-1] + Sodd[i+1, N-1];
```

```
    TEven = pfEven[i-1] + pfOdd[N-1] - pfOdd[i];
```

```
    Sodd[0, i-1] + SEven[i+1, N-1];
```

```
    Todd = pfOdd[i-1] + pfEven[N-1] - pfEven[i];
```

```
    if (TEven == Todd) { // Special Index
```

```
        cnt++;
```

```
    }
```

```
}
```

$$T.C = O(N + N + N) = O(N)$$

$$S.C = O(N + N) = O(N)$$

Edge case when $i = 0$

Pseudo Code

//construct pfEven[]

//construct pfOdd[]

int cnt=0;

for(i=0; i < N; i++){

if(i==0){

 T_{even} = pfOdd[N-1] - pfOdd[i];

 T_{odd} = pfEven[N-1] - pfEven[i];

}

else {

 S_{even}[0, i-1] + S_{odd}[i+1, N-1];

 T_{even} = pfEven[i-1] + pfOdd[N-1] - pfOdd[i];

 S_{odd}[0, i-1] + S_{even}[i+1, N-1];

 T_{odd} = pfOdd[i-1] + pfEven[N-1] - pfEven[i];

if (T_{even} == T_{odd}) { //Special Index

 cnt++;

}

}

}

T.C = O(N)

S.C = O(N)